

Final Presentation

Lightweight CNN Implementation for MNIST Digit Classification

m5291005 Kyoji Awaki

2025/9/12

- ❑ Motivation and Purpose
- ❑ Explanation of your application
 - Functions with a figure
 - Explanation on how do you create your Verilog HDL model
- ❑ Explanation of your architecture with a figure
- ❑ Design flow and used tools with a figure
- ❑ Simulation results
- ❑ Evaluation results with a table or graph
- ❑ Demonstration using an FPGA board
- ❑ Summary
- ❑ The time which you spend for your project work

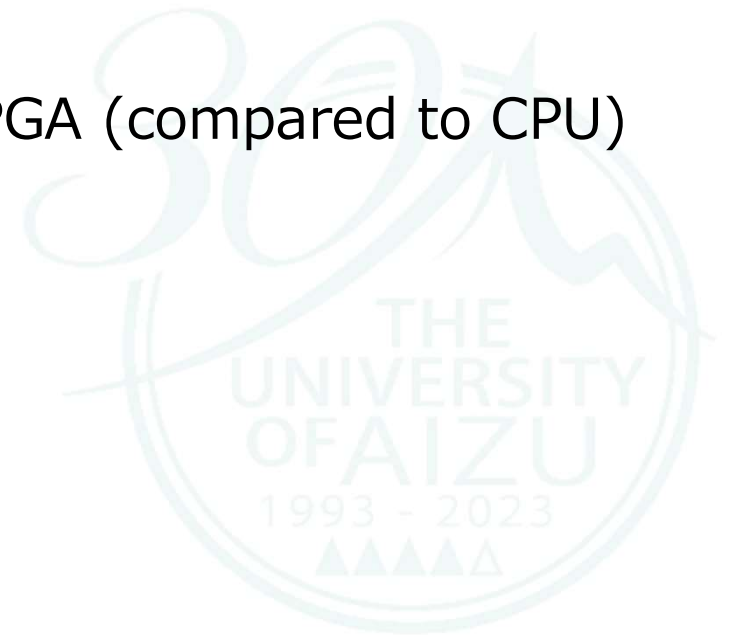


□ Motivation

- CNNs are widely used for object detection and classification in IoT systems.
- CNNs require numerous convolution operations, resulting in a massive computational load.

□ Purpose

- Accelerating CNN inference processing using FPGA (compared to CPU)

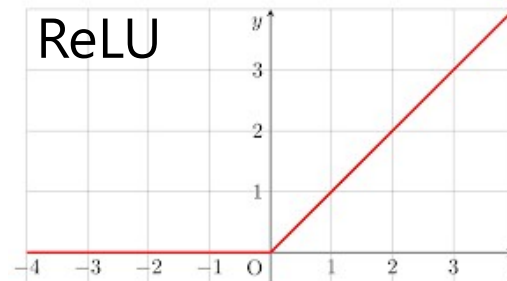


Target Application - CNN

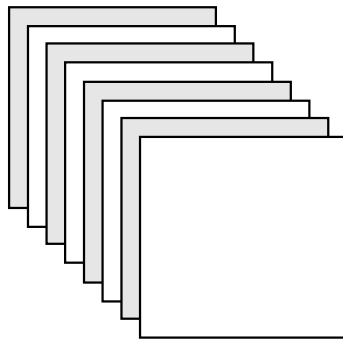
4

Conv

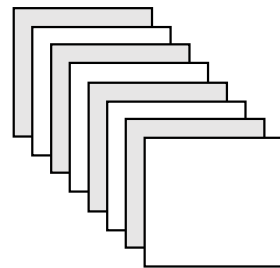
$$\sum_{i=-1}^1 \sum_{j=-1}^1 \text{Image}(x+i, y+j) \cdot \text{Filter}(i+1, j+1) \rightarrow$$



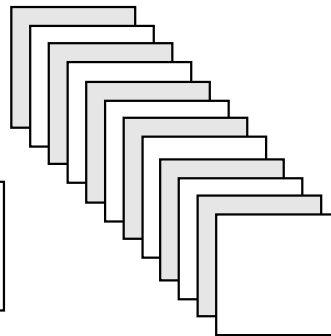
Input: 28x28
grayscale image



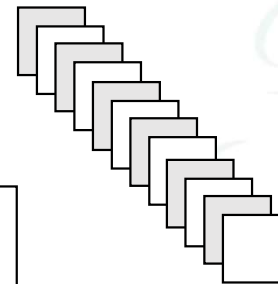
Conv+ReLU



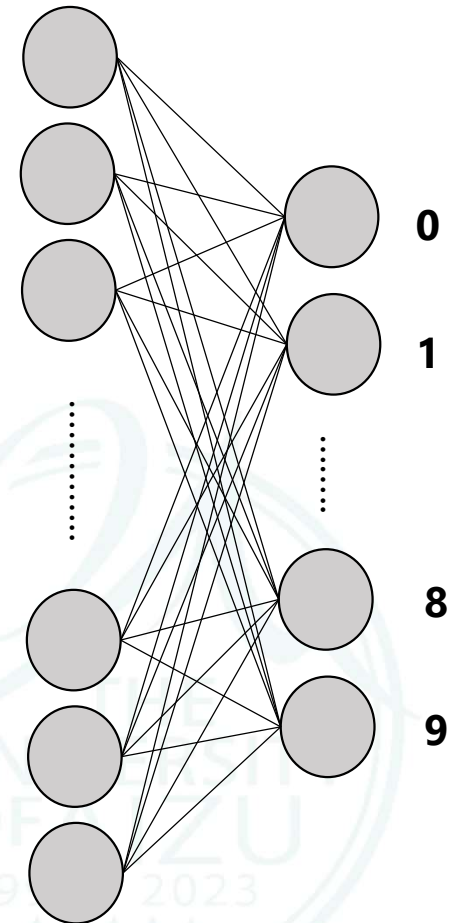
MaxPool



Conv+ReLU



MaxPool



Output: Class labels
from 0 to 9

Model Structure

5

Layer	Input Size	Operation	Output Size
Input	$1 \times 28 \times 28$	Grayscale image	$1 \times 28 \times 28$
Conv1	$1 \times 28 \times 28$	Conv2d($1 \rightarrow 4$, kernel= 3×3 , stride=1)	$4 \times 26 \times 26$
Pool1	$4 \times 26 \times 26$	MaxPool(2×2)	$4 \times 13 \times 13$
Conv2	$4 \times 13 \times 13$	Conv2d($4 \rightarrow 8$, kernel= 3×3 , stride=1)	$8 \times 11 \times 11$
Pool2	$8 \times 11 \times 11$	MaxPool(2×2)	$8 \times 5 \times 5$
Flatten	$8 \times 5 \times 5$	–	200
FC (Output)	200	Fully Connected $\rightarrow 10$	10

Training Specifications

6

Item	Content
Quantization Bit-Width	8-bit
Dataset	MNIST
Data Split	Train: 55,000 / Val: 5,000 / Test: 10,000
Network	Conv (c: 1→4, k=3) → MaxPool(2) → Conv (c: 4→8, k=3) → MaxPool(2) → FC (200→10)
Training	Adam (lr=1e-3, wd=1e-4), batch=128, epochs=10
QAT	1-epoch warm-up, then enable quantization
Results	Test Loss $\approx$ 0.0588, Accuracy $\approx$ 98.15% (after 10 epochs)

Target Board

7

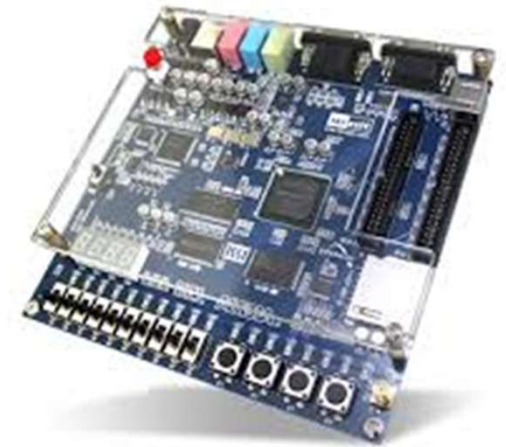
❑ Terasic DE2-115 (Cyclone IV E EP4CE115)

■ Performance (Key Specs)

- ❑ 114,480 logic elements (LEs)
- ❑ ~3.9 Mbit embedded memory
- ❑ 266 embedded 18×18 multipliers (DSP blocks)
- ❑ 4 PLLs, 528 user I/Os
- ❑ On-board memory: 128 MB SDRAM, 2 MB SRAM, 8 MB Flash
- ❑ Rich peripherals: Ethernet, VGA, Audio codec, USB, SD card

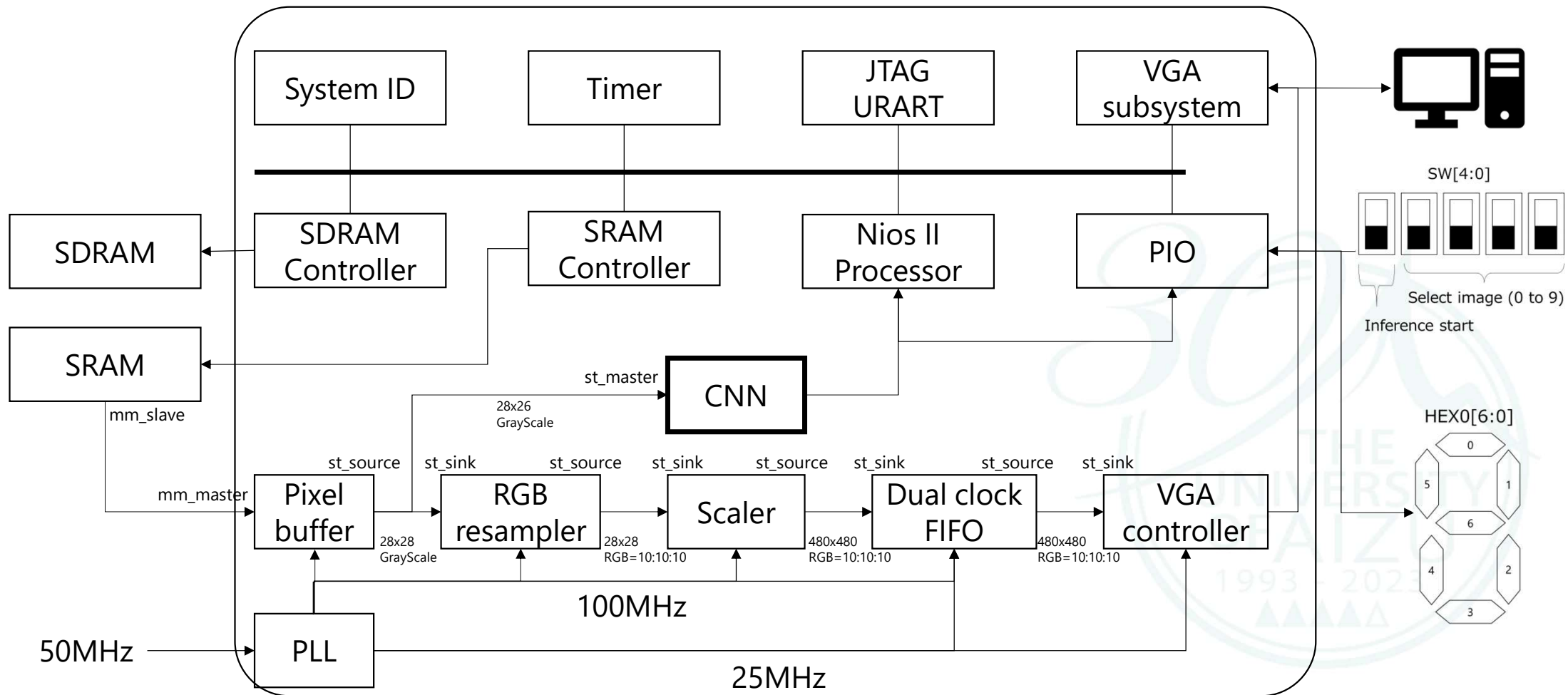
■ Reasons for Selection

- ❑ Rich set of on-board peripherals reduces need for external hardware

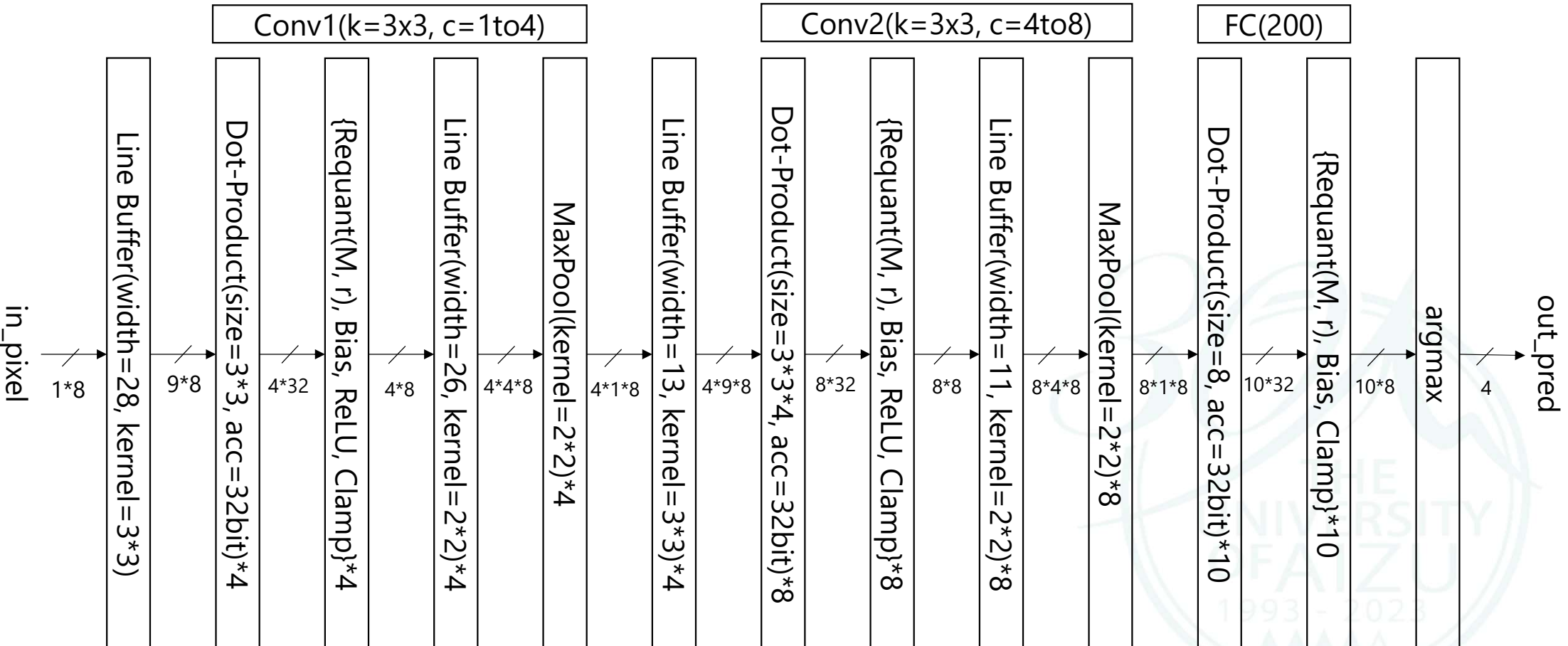


8

Cyclone IV FPGA

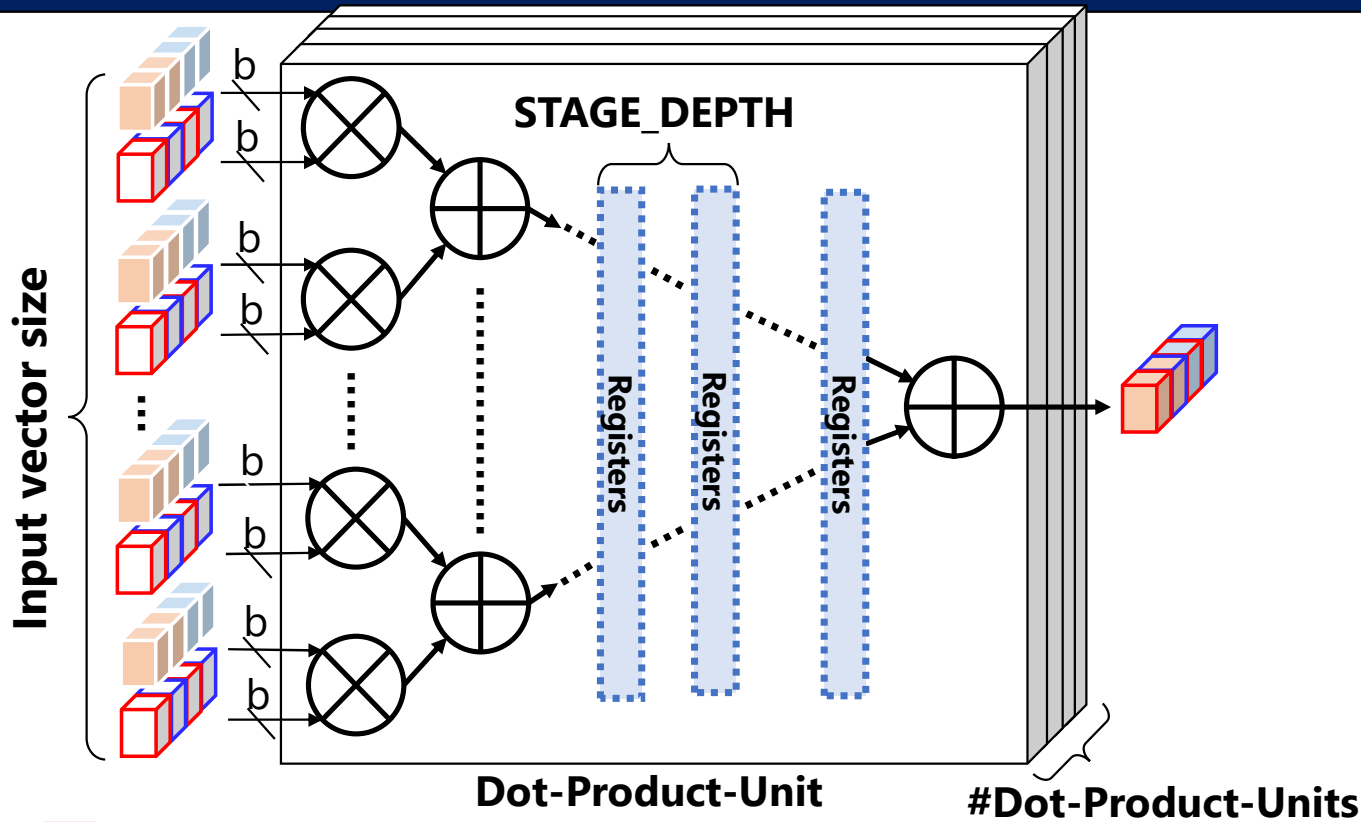


CNN Architecture



Dot Product Unit (Convolution Operations)

10



$$\begin{aligned}
 o_1 &= a_{1,1} \cdot w_{1,1} + a_{1,2} \cdot w_{1,2} + \dots + a_{1,n-1} \cdot w_{1,n-1} + a_{1,n} \cdot w_{1,n} \\
 o_2 &= a_{1,1} \cdot w_{2,1} + a_{1,2} \cdot w_{2,2} + \dots + a_{1,n-1} \cdot w_{2,n-1} + a_{1,n} \cdot w_{2,n} \\
 o_3 &= a_{2,1} \cdot w_{1,1} + a_{2,2} \cdot w_{1,2} + \dots + a_{2,n-1} \cdot w_{1,n-1} + a_{2,n} \cdot w_{1,n} \\
 o_4 &= a_{2,1} \cdot w_{2,1} + a_{2,2} \cdot w_{2,2} + \dots + a_{2,n-1} \cdot w_{2,n-1} + a_{2,n} \cdot w_{2,n}
 \end{aligned}$$

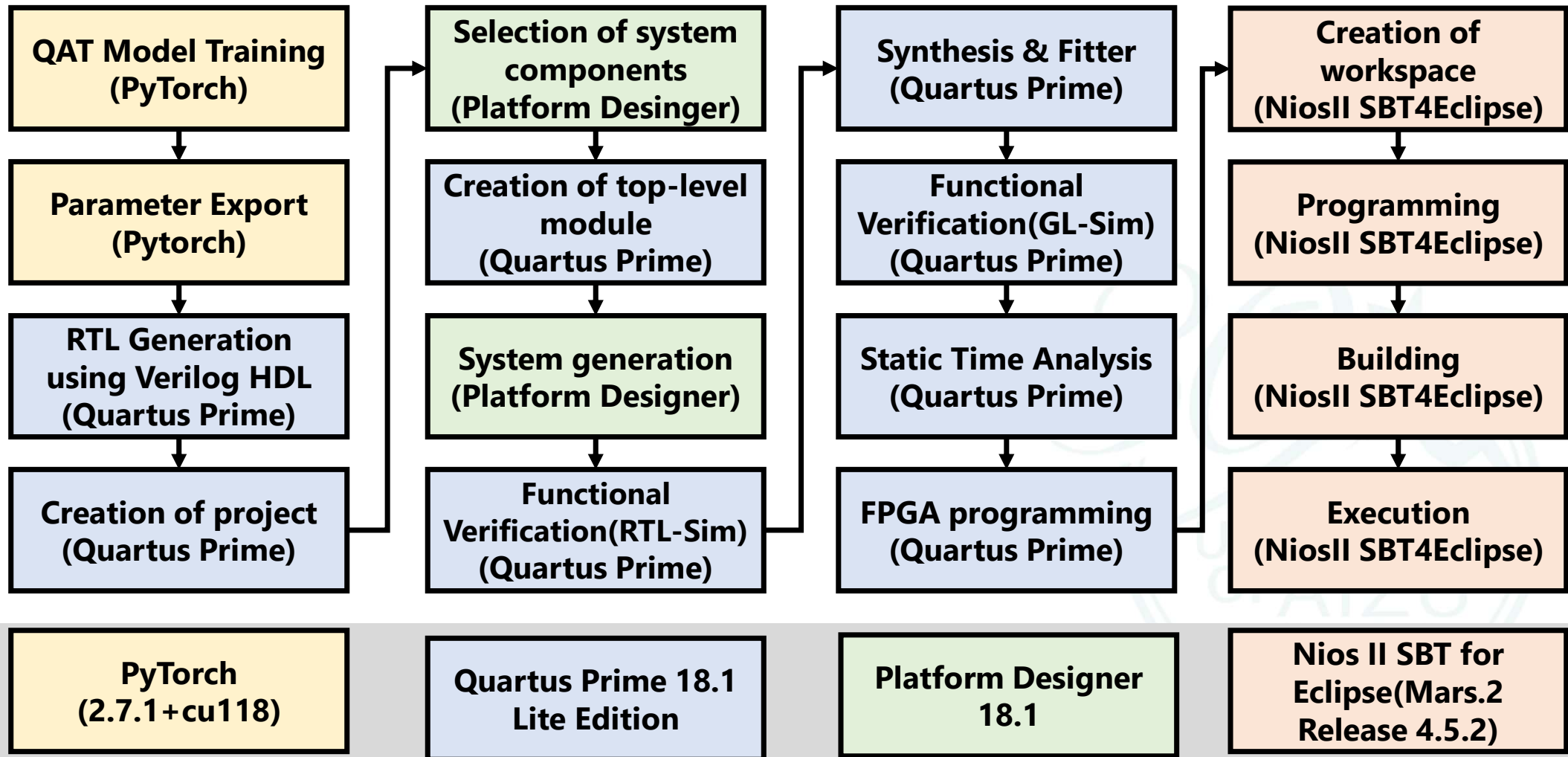
```

module DotProductUnit #(
    parameter integer N          = 3*3*1, // ch1
    parameter integer ACT_WIDTH  = 8,     // x(8bit)
    parameter integer WEIGHT_WIDTH = 8,    // w(8bit)
    parameter integer STAGE_DEPTH = 3,
    parameter integer MULT_W      = 16,
    parameter integer ACC_W       = 32     // acc(32bit)
)()
    input wire clk,
    input wire reset,
    input wire in_valid,
    input wire wen,
    input wire [WEIGHT_WIDTH*N-1:0] w_vec,
    input wire [ACT_WIDTH*N-1:0] a_vec,
    output wire [ACC_W-1:0] out,
    output wire out_valid

    wire [MULT_W*N-1:0] mult_vec;
    genvar k;
    generate
        for (k = 0; k < N; k = k + 1) begin
            assign mult_vec[MULT_W*k +: MULT_W]
                = $signed(w_vec[WEIGHT_WIDTH*k +: WEIGHT_WIDTH])
                  * $signed(a_vec[ACT_WIDTH*k +: ACT_WIDTH]);
        end
    endgenerate
    AdderTree adder_tree
        (.clk(clk), .reset(reset), .wen(wen),
         .idata(mult_vec), .odata(out));
endmodule
    
```

Design Flow and Used Tools

11



CPU Baseline

12

- ❑ OS: Windows 11 Pro 24H2
- ❑ CPU: Intel Core i7-13700KF
- ❑ Memory: 32 GB

Build	samples	total [ms]	avg [μ s/sample]	Correct answers/total
O3	8,077	120.823	15	7,982 / 8,077 (98.82%)
O2	8,077	259.882	32	7,982 / 8,077 (98.82%)

Resource Utilization(Core Circuit)

13

Resource	Usage	Available	Utilization
Logic Elements (LEs)	30,717	114,480	27 %
Registers	20,469	–	–
Pins	16	529	3 %
Virtual Pins	0	–	–
Memory Bits	739	3,981,312	< 1 %
Embedded Multipliers (9-bit)	251	532	47 %
PLLs	0	4	0 %

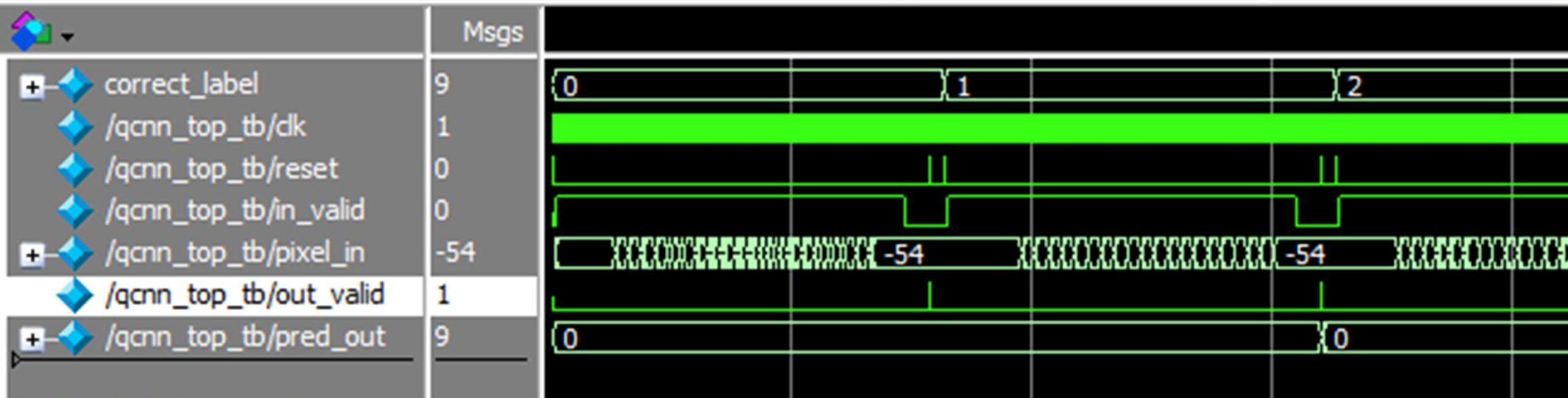
Clock frequency (Core Circuit)

14

Item	Value / Observation
Required Clock Period	10.0 ns (100 MHz)
Slack	+0.001 ns
Worst Path (From → To)	Conv1out: Requantization multiply ($ acc \times scale \rightarrow DSP\ result$)
Bottleneck	DSP 32×32 multiplier

Gate-Level-Simulation

15



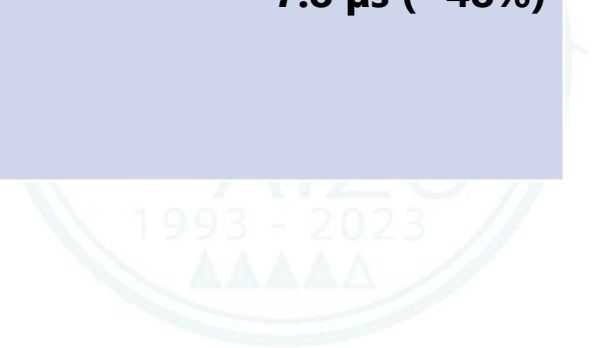
```
# Loaded img_mem.hex.
# Output data for image: label=0, pred=0
# Output data for image: label=1, pred=1
# Output data for image: label=2, pred=2
# Output data for image: label=3, pred=3
# Output data for image: label=4, pred=4
# Output data for image: label=5, pred=5
# Output data for image: label=6, pred=6
# Output data for image: label=7, pred=7
# Output data for image: label=8, pred=8
# Output data for image: label=9, pred=9
```

```
# ---- Summary ----
# Images measured      : 10
# Avg latency (ns)    : 7800.000000
# Throughput (img/s)  : 128205.128205
# Accuracy             : 100%
```

Execution time

16

Metric	CPU (O3)	FPGA (Simulation)
Latency / Image	15 μ s (baseline)	7.8 μ s (–48%)



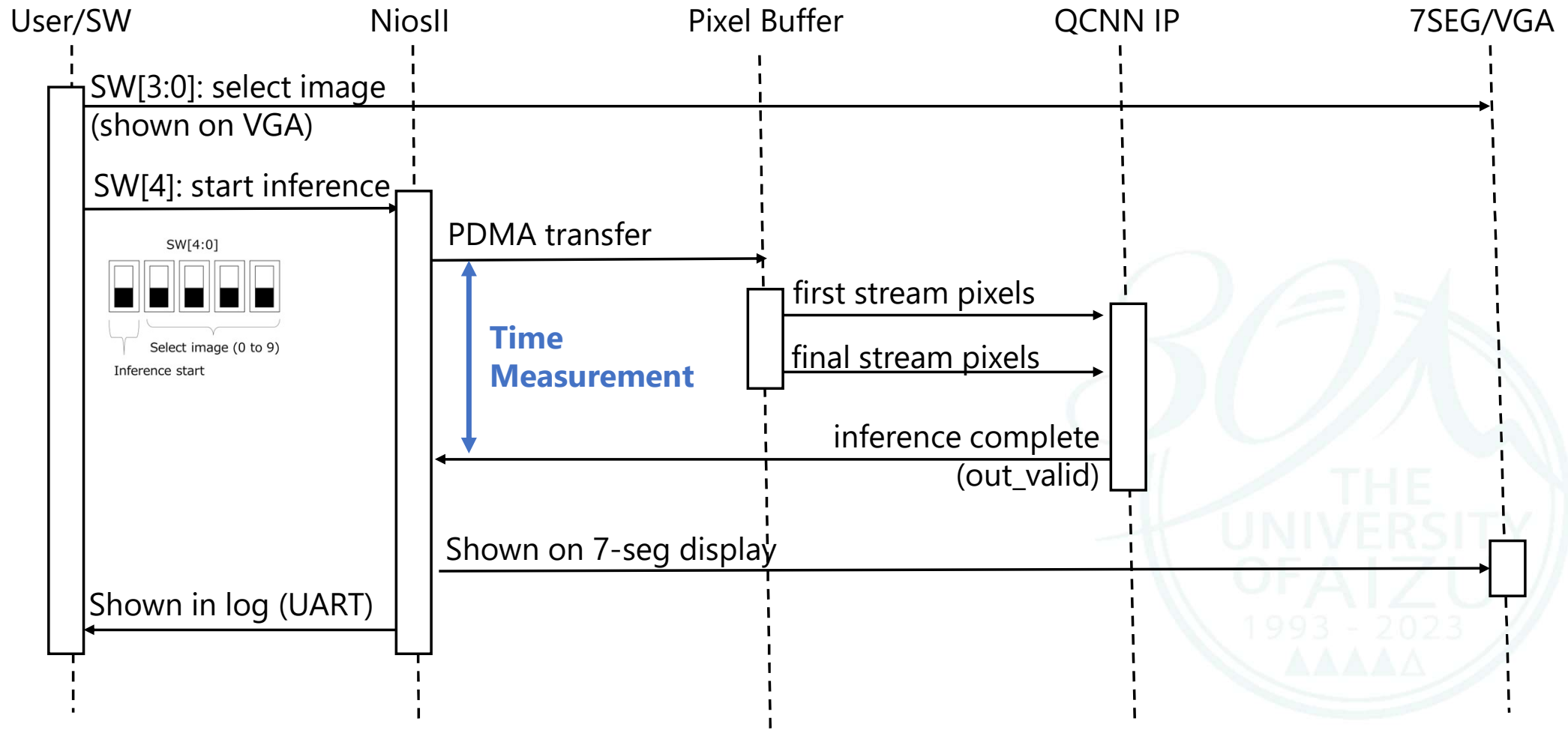
Resource Utilization(Overall)

17

Resource	Usage	Available	Utilization
Logic Elements (LEs)	34,370	114,480	30%
Registers	23,781	—	—
Pins	154	529	29%
Virtual Pins	0	—	—
Memory Bits	34,851	3,981,312	<1%
Embedded Multipliers (9-bit)	251	532	47%
PLLs	1	4	25%

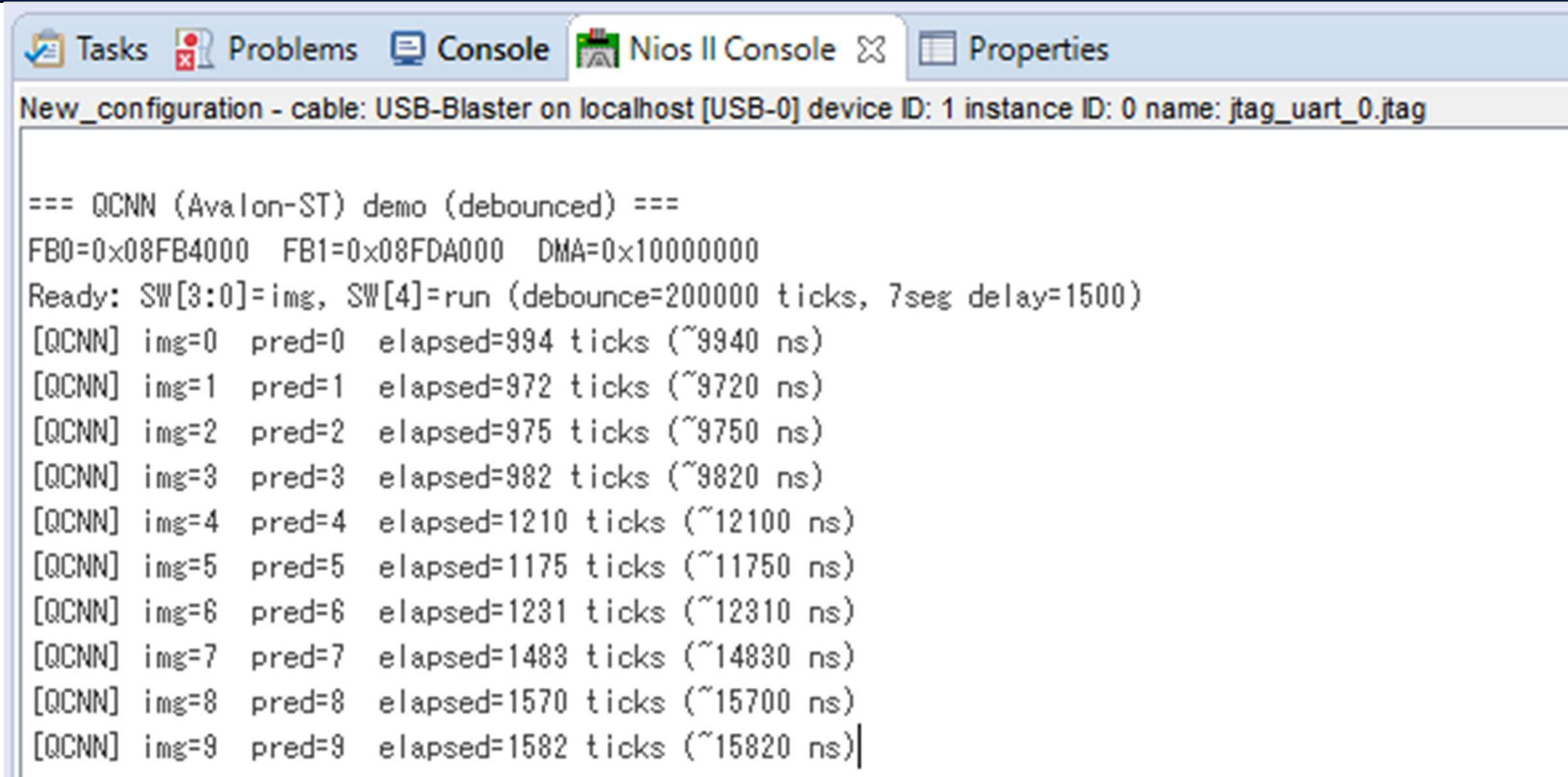
Demonstration Processing Flow

18



Result of Demo

19



The screenshot shows the Nios II IDE interface with the 'Nios II Console' tab selected. The console output displays the results of a QCNN (Avalon-ST) demo. The output includes initialization parameters like FB0, FB1, and DMA, followed by a 'Ready' message and a series of 10 test results. Each result line shows the input image (img), the predicted value (pred), and the elapsed time in ticks and nanoseconds.

```
Tasks Problems Console Nios II Console Properties
New_configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtag_uart_0.jtag

=== QCNN (Avalon-ST) demo (debounced) ===
FB0=0x08FB4000 FB1=0x08FDA000 DMA=0x10000000
Ready: SW[3:0]=img, SW[4]=run (debounce=200000 ticks, 7seg delay=1500)
[QCNN] img=0 pred=0 elapsed=994 ticks (~9940 ns)
[QCNN] img=1 pred=1 elapsed=972 ticks (~9720 ns)
[QCNN] img=2 pred=2 elapsed=975 ticks (~9750 ns)
[QCNN] img=3 pred=3 elapsed=982 ticks (~9820 ns)
[QCNN] img=4 pred=4 elapsed=1210 ticks (~12100 ns)
[QCNN] img=5 pred=5 elapsed=1175 ticks (~11750 ns)
[QCNN] img=6 pred=6 elapsed=1231 ticks (~12310 ns)
[QCNN] img=7 pred=7 elapsed=1483 ticks (~14830 ns)
[QCNN] img=8 pred=8 elapsed=1570 ticks (~15700 ns)
[QCNN] img=9 pred=9 elapsed=1582 ticks (~15820 ns)
```

Metric	CPU (O3)	Core (Simulation)	FPGA(Actual Device)
Latency / Image	15 μ s (baseline)	7.8 μ s (-48%)	 12.2 μ s(-19%)

- I/O Latency
- Memory Access
- Inter-board Communication and Peripheral Circuit Constraints
- Nios II processor overhead instability

- ❑ I implemented CNN on FPGA
 - GL-Sim: 2 times faster compared with Intel Core i7-13700KF
 - Actual Hardware: 1.2 times faster
- ❑ Future work
 - Improve accuracy of QAT models with lower bit-width



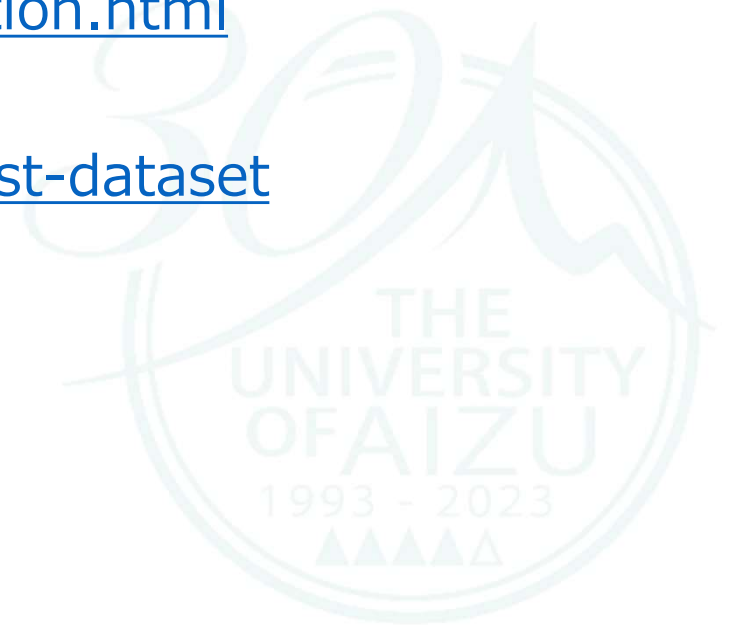
Time Spent on the Project

22

- ❑ Decision + Initial report: 16 h
- ❑ Processor eval: 12 h
- ❑ Architecture consideration: 8 h
- ❑ Design work: 20 h
- ❑ Intermediate presentation: 10 h
- ❑ Optimization & evaluation: 15 h
- ❑ Demonstration: 15 h
- ❑ Final presentation: 10 h
- =====
- ❑ Total = 106 h



- [1] Course materials of the Integrated Exercise on System2
- [2] Intel official documents
 - <https://www.intel.co.jp/content/www/jp/products/details/fpga.html>
- [3] PyTorch: Quantization Aware Training (QAT)
 - <https://docs.pytorch.org/docs/stable/quantization.html>
- [4] MNIST datasets
 - <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>



Thank you for your attention.

Lightweight CNN Implementation for MNIST Digit Classification

m529105 Kyoji Awaki

2025/9/12